

Plant Disease Detection Notebook - Complete Marathi Explanation

पहिला Notebook: Data Preprocessing (डेटा प्रीप्रोसेसिंग)

भाग 1: Library Import (लायब्ररी इम्पोर्ट)

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tqdm import tqdm
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.utils import image_dataset_from_directory
```

मराठी स्पष्टीकरण:

- `import os` - फोल्डर आणि फाइल पाथ हाताळण्यासाठी (operating system कमांड)
- `import cv2` - इमेज प्रोसेसिंग साठी (OpenCV library)
- `import numpy as np` - मॅथमेटिकल कॅल्क्युलेशन आणि अॅर्रे ऑपरेशनसाठी
- `import matplotlib.pyplot as plt` - इमेजेस दाखवण्यासाठी (व्हिज्युअलायझेशन)
- `import tensorflow as tf` - न्यूरल नेटवर्क तयार करण्यासाठी (मुख्य लायब्ररी)
- `from tqdm import tqdm` - प्रोग्रेस बार दाखवण्यासाठी
- `from sklearn.model_selection import train_test_split` - डेटा ट्रेन आणि टेस्ट मध्ये विभाजित करण्यासाठी
- `from tensorflow.keras.utils import to_categorical` - लेबल्स ला one-hot encoding मध्ये रूपांतरित करण्यासाठी
- `from tensorflow.keras.utils import image_dataset_from_directory` - फोल्डर मधून थेट इमेजेस लोड करण्यासाठी

भाग 2: Dataset Path Setting (डेटासेट पाथ सेट करणे)

```
dataset_path = "../PlantVillage"
classes = ["Pepper_bell_Bacterial_spot", "Pepper_bell_healthy"]
BATCH_SIZE = 32
IMG_HEIGHT = 128
IMG_WIDTH = 128
```

मराठी स्पष्टीकरण:

- `dataset_path = "../PlantVillage"` - डेटासेट चा पाथ. `..` म्हणजे एक फोल्डर वर (नोटबुक पासून)
- `classes` - दोन क्लासेस: बॅक्टेरियल स्पॉट (रोगट) आणि हेल्दी (निरोगी)
- `BATCH_SIZE = 32` - एकाच वेळी 32 इमेजेस प्रोसेस करेल (मेमरीसाठी चांगले)

- `IMG_HEIGHT = 128, IMG_WIDTH = 128` - सर्व इमेजेस 128×128 pixels मध्ये रीसाइज करतो
-

भाग 3: Dataset Loading (डेटासेट लोड करणे)

```
dataset = image_dataset_from_directory(  
    dataset_path,  
    seed=123,  
    image_size=(IMG_HEIGHT, IMG_WIDTH),  
    batch_size=BATCH_SIZE  
)  
class_names = dataset.class_names
```

मराठी स्पष्टीकरण:

- `image_dataset_from_directory` - फोल्डर स्ट्रक्चर वरून ऑटोमॅटिक डेटा लोड करतो
 - `seed=123` - रँडम सॅम्पलिंग कंट्रोल करतो (प्रत्येक वेळी सारखे रिझल्ट)
 - `image_size=(128,128)` - सर्व इमेजेस 128×128 मध्ये resize
 - `batch_size=32` - 32 इमेजेस चा ग्रुप (batch)
 - `class_names` - फोल्डरच्या नावांवरून क्लासेस ओळखतो
-

भाग 4: Visualize Images (इमेजेस दाखवणे)

```
plt.figure(figsize=(10, 10))  
for images, labels in dataset.take(1):  
    for i in range(9):  
        plt.subplot(3, 3, i + 1)  
        plt.imshow(images[i].numpy().astype("uint8"))  
        plt.title(class_names[labels[i]])  
        plt.axis("off")  
plt.show()
```

मराठी स्पष्टीकरण:

- `plt.figure(figsize=(10,10))` - 10×10 इंच चा फिगर तयार करतो
 - `dataset.take(1)` - डेटासेट मधला पहिला batch घेतो (32 इमेजेस)
 - `plt.subplot(3,3,i+1)` - 3×3 ग्रिड तयार करतो (एकूण 9 इमेजेस)
 - `plt.imshow()` - इमेज प्रिंट करतो
 - `.numpy().astype("uint8")` - टेन्सर ला numpy array मध्ये बदलतो
 - `plt.title()` - इमेज वर क्लासचे नाव दाखवतो
 - `plt.axis("off")` - x-y अक्ष लपवतो
-

भाग 5: Dataset Information (डेटासेट माहिती)

```
print("Class Names :", class_names)  
print("Number of Classes :", len(class_names))  
for image_batch, labels_batch in dataset.take(1):  
    print("Image Batch Shape :", image_batch.shape)  
    print("Label Batch Shape :", labels_batch.shape)
```

मराठी स्पष्टीकरण:

- क्लासेसची नावे आणि संख्या दाखवते
 - `image_batch.shape` - (32, 128, 128, 3) 32 इमेजेस, 128×128 pixels, 3 कलर चॅनेल (RGB)
 - `labels_batch.shape` - (32,) 32 लेबल्स
-

भाग 6: Train-Validation Split (ट्रेन-व्हॅलिडेशन विभाजन)

```
dataset_size = len(dataset)
train_size = int(0.8 * dataset_size)
train_dataset = dataset.take(train_size)
val_dataset = dataset.skip(train_size)
```

मराठी स्पष्टीकरण:

- `dataset_size` - एकूण किती batches आहेत
 - `train_size = 0.8 * dataset_size` - 80% batches ट्रेनिंगसाठी
 - `dataset.take(train_size)` - पहिले 80% batches ट्रेनिंग सेट
 - `dataset.skip(train_size)` - उरलेले 20% batches व्हॅलिडेशन सेट
-

भाग 7: Performance Optimization (परफॉर्मन्स ऑप्टिमायझेशन)

```
AUTOTUNE = tf.data.AUTOTUNE
train_dataset =
train_dataset.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_dataset = val_dataset.cache().prefetch(buffer_size=AUTOTUNE)
```

मराठी स्पष्टीकरण:

- `.cache()` - डेटा मेमरी मध्ये स्टोअर करतो (वेगाने लोड होते)
 - `.shuffle(1000)` - 1000 इमेजेस रँडमली शफल करतो (ऑर्डर बदलतो)
 - `.prefetch()` - पुढचा batch लोड करतो (मॉडेल प्रशिक्षण घेत असतानाच)
 - `AUTOTUNE` - टेन्सरफ्लो ऑटोमॅटिकली बेस्ट सेटिंग्ज निवडतो
-

भाग 8: Normalization (नॉर्मलायझेशन)

```
normalization_layer = tf.keras.layers.Rescaling(1./255)
normalized_train_dataset = train_dataset.map(lambda x, y:
(normalization_layer(x), y))
```

मराठी स्पष्टीकरण:

- `Rescaling(1./255)` - पिक्सल व्हॅल्यू 0-255 वरून 0-1 मध्ये बदलते
 - का? - न्यूरल नेटवर्क लहान संख्यांवर लवकर शिकते
 - `.map()` - प्रत्येक बॅच ला नॉर्मलायझेशन लागू करतो
-

भाग 9: Data Augmentation (डेटा ऑगमेंटेशन)

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.2),
```

```
tf.keras.layers.RandomZoom(0.2),  
])
```

मराठी स्पष्टीकरण:

- `RandomFlip` - इमेज उलटी-सुलटी करतो (आरशातील प्रतिमा)
 - `RandomRotation(0.2)` - इमेज 20% पर्यंत फिरवतो
 - `RandomZoom(0.2)` - इमेज 20% पर्यंत झूम करतो
 - **उद्देश:** मर्यादित डेटा पासून नवीन वैरायटी तयार करणे (ओव्हरफिटिंग टाळते)
-

दुसरा Notebook: Model Building (मॉडेल तयार करणे)

भाग 1: CNN Model Architecture (सीएनएन मॉडेल आर्किटेक्चर)

```
model = models.Sequential([  
    data_augmentation,  
    layers.Conv2D(32, (3,3), activation='relu', input_shape=  
(128,128,3)),  
    layers.MaxPooling2D((2,2)),  
    layers.Conv2D(64, (3,3), activation='relu'),  
    layers.MaxPooling2D((2,2)),  
    layers.Conv2D(128, (3,3), activation='relu'),  
    layers.MaxPooling2D((2,2)),  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dropout(0.5),  
    layers.Dense(2, activation='softmax')  
])
```

मराठी स्पष्टीकरण - प्रत्येक लेयर:

1. Data Augmentation Layer:

- डेटा ऑगमेंटेशन लागू करतो

2. Conv2D(32, (3,3), activation='relu'):

- `32` - 32 फिल्टर्स (फीचर डिटेक्टर)
- `(3,3)` - फिल्टर साइज (3×3 pixels)
- `activation='relu'` - नॉन-लिनियरिटी जोडते (निगेटिव नंबर 0 करते)
- **काम:** इमेज मधून एज, कॉर्नर, टेक्सचर डिटेक्ट करते

3. MaxPooling2D((2,2)):

- 2×2 विंडो मधून मॅक्सिमम व्हॅल्यू निवडते
- **काम:** इमेजचा साइज अर्धा करते (128→64, 64→32, 32→16)
- **फायदा:** कॉम्प्युटेशन कमी होते, ओव्हरफिटिंग कमी होते

4. Flatten():

- 2D (16×16×128) ला 1D (32768) मध्ये बदलते
- **काम:** कन्व्होल्यूशन लेयर नंतर डेन्स लेयर ला कनेक्ट करते

5. Dense(128, activation='relu'):

- फुली कनेक्टेड लेयर (128 न्यूरॉन्स)
- **काम:** फीचर्स वरून क्लास डिसिजन घेते

6. Dropout(0.5):

- 50% न्यूरॉन्स रँडमली बंद करते
- **काम:** ओव्हरफिटिंग टाळते

7. Dense(2, activation='softmax'):

- आउटपुट लेयर (2 न्यूरॉन्स - दोन क्लासेस)
- `softmax` - प्रोबॅबिलिटी वितरीत करते (एकूण = 1)
- **उदा:** [0.9, 0.1] म्हणजे 90% क्लास 0, 10% क्लास 1

भाग 2: Model Compilation (मॉडेल कंपाइल करणे)

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

मराठी स्पष्टीकरण:

- `optimizer='adam'` - वजन अपडेट करण्याची अल्गोरिदम (सर्वात लोकप्रिय)
 - काम: त्रुटी कमी करण्यासाठी weights बदलते
- `loss='sparse_categorical_crossentropy'` - एरर कॅल्क्युलेट करते
 - का? मल्टी-क्लास क्लासिफिकेशन साठी (2+ क्लासेस)
- `metrics=['accuracy']` - अक्युरसी मोजते

भाग 3: Training (प्रशिक्षण)

```
EPOCHS = 20  
history = model.fit(  
    train_dataset,  
    validation_data=val_dataset,  
    epochs=EPOCHS,  
    callbacks=[early_stopping]  
)
```

मराठी स्पष्टीकरण:

- `EPOCHS = 20` - संपूर्ण डेटासेट वर 20 वेळा प्रशिक्षण
- `fit()` - मॉडेल ट्रेन करते
- `train_dataset` - ट्रेनिंग डेटा
- `validation_data` - व्हॅलिडेशन डेटा (प्रत्येक epoch नंतर चेक करते)
- `callbacks=[early_stopping]` - जर सुधारणा थांबली तर लवकर थांबवते

Training Process (समजून घ्या):

Epoch 1: Model शिकते (अक्युरसी 40%)
Epoch 2: थोडे सुधारते (50%)
Epoch 3: अजून सुधारते (60%)
.
.
Epoch 15: अक्युरसी 95% पर्यंत पोहोचते

भाग 4: Early Stopping (लवकर थांबवणे)

```
early_stopping = tf.keras.callbacks.EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

मराठी स्पष्टीकरण:

- `monitor='val_loss'` - व्हॅलिडेशन लॉस बघतो
 - `patience=5` - जर 5 epochs मध्ये सुधारणा नाही झाली तर थांबवते
 - `restore_best_weights=True` - जेव्हा लॉस सर्वात कमी होता तेव्हाचे weights परत आणते
 - **उद्देश:** ओव्हरफिटिंग आणि वेळ वाया जाण्यापासून वाचवते
-

भाग 5: Evaluation (मूल्यांकन)

```
test_loss, test_accuracy = model.evaluate(val_dataset)
```

मराठी स्पष्टीकरण:

- `evaluate()` - व्हॅलिडेशन डेटा वर मॉडेल चाचणी करते
 - `test_loss` - मॉडेलची त्रुटी (0 च्या जवळ चांगले)
 - `test_accuracy` - किती टक्के बरोबर उत्तरे (1 च्या जवळ चांगले)
-

भाग 6: Prediction (भविष्यवाणी)

```
img = image.load_img(image_path, target_size=(128, 128))  
img_array = image.img_to_array(img)  
img_array = np.expand_dims(img_array, axis=0)  
img_array = img_array / 255.0  
prediction = model.predict(img_array)  
predicted_class = class_names[np.argmax(prediction)]
```

मराठी स्पष्टीकरण (चरणबद्ध):

1. `load_img(path, target_size=(128,128))`
 - इमेज लोड करते आणि 128×128 मध्ये रीसाइज करते
2. `img_to_array(img)`
 - PIL इमेज ला numpy array मध्ये बदलते
 - शेप: (128, 128, 3)
3. `np.expand_dims(img_array, axis=0)`

- नवीन डायमेंशन जोडते
 - शेप: (1, 128, 128, 3) → batch dimension
4. `/ 255.0`
- नॉर्मलायझेशन (0-255 → 0-1)
5. `model.predict(img_array)`
- आउटपुट: `[[0.85, 0.15]]`
 - पहिली व्हॅल्यू: क्लास 0 ची प्रोबॅबिलिटी (Bacterial Spot)
 - दुसरी व्हॅल्यू: क्लास 1 ची प्रोबॅबिलिटी (Healthy)
6. `np.argmax(prediction)`
- $0.85 > 0.15$ म्हणून index 0 निवडते
 - म्हणजे "Pepper_bell_Bacterial_spot"
-

प्रशिक्षणाचे Visualization Graphs

Accuracy Graph:

```
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
```

- **हिरवी रेषा** - ट्रेनिंग अक्युरसी (शिकत असतानाची)
- **केसरी रेषा** - व्हॅलिडेशन अक्युरसी (नवीन डेटा वरची)
- चांगले मॉडेल: दोन्ही रेषा जवळ असतात आणि वर जातात

Loss Graph:

```
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
```

- **लॉस कमी होत गेला पाहिजे**
- जर val_loss वाढू लागला तर ओव्हरफिटिंग

Key Takeaways (महत्त्वाचे मुद्दे)

1. Data Flow:

```
Raw Images (PlantVillage/)
→ Dataset Loading
→ Normalization (0-1)
→ Augmentation
→ CNN Training
→ Model Saving
→ Prediction
```

2. Why 128×128?

- मोठी इमेज (512×512) मेमरी जास्त घेते
- लहान इमेज (64×64) फीचर्स हरवते

- 128×128 हा चांगला समतोल

3. Batch Size = 32 का?

- 16 = स्लो पण अचूक
- 32 = बॅलेंस
- 64 = फास्ट पण कमी अचूक

4. Overfitting लक्षणे:

- ट्रेनिंग अक्युरसी 99%, पण व्हॅलिडेशन अक्युरसी 70%
- मॉडेल फक्त ट्रेनिंग इमेजेस "मेमोराईज" करते

हा तुमचा संपूर्ण प्रोजेक्ट समजण्यासाठी पुरेसा मराठी स्पष्टीकरण आहे. काही शंका असल्यास विचारा! 🌱

Plant Disease Detection Program - Complete Marathi Explanation

तुमच्या `final_model.ipynb` प्रोग्रामची संपूर्ण मराठीत समज देणारी माहिती खालीलप्रमाणे आहे.

पहिला भाग: Libraries Import (लायब्ररीज इम्पोर्ट करणे)

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
import matplotlib.pyplot as plt
from sklearn.utils.class_weight import compute_class_weight
import os
```

मराठी अर्थ:

- `tensorflow` - न्यूरल नेटवर्क तयार करण्यासाठी मुख्य लायब्ररी
- `layers, models` - CNN चे विविध स्तर (convolution, pooling, dense) तयार करण्यासाठी
- `ImageDataGenerator` - इमेजेस लोड करण्यासाठी आणि ऑगमेंटेशन करण्यासाठी
- `numpy` - न्यूमेरिकल ॲर्रे ऑपरेशन्ससाठी
- `matplotlib.pyplot` - आलेख (graphs) आणि इमेजेस दाखवण्यासाठी
- `compute_class_weight` - असंतुलित डेटासाठी क्लास वेट्स कॅल्क्युलेट करण्यासाठी
- `os` - फाइल सिस्टम ऑपरेशन्ससाठी

दुसरा भाग: Parameters (पॅरामीटर्स)

```
IMG_HEIGHT, IMG_WIDTH = 128, 128
BATCH_SIZE = 16
DATASET_PATH = "./PlantVillage"
```

मराठी अर्थ:

- `IMG_HEIGHT, IMG_WIDTH = 128, 128` - सर्व इमेजेस 128x128 पिक्सेलमध्ये रीसाइज केल्या जातील
- `BATCH_SIZE = 16` - एकाच वेळी 16 इमेजेस प्रोसेस केल्या जातील (मेमरी कमी वापरते)
- `DATASET_PATH = "./PlantVillage"` - डेटासेट ज्या फोल्डरमध्ये आहे तो पाथ

BATCH_SIZE 16 का? - 16 चा batch small आहे, त्यामुळे:

- कमी मेमरी लागते
- मॉडेल अधिक स्थिर (stable) प्रशिक्षण घेते
- प्रत्येक batch नंतर weights अपडेट होतात

तिसरा भाग: Data Generators (डेटा जनरेटर)

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    horizontal_flip=True,
    zoom_range=0.2,
    fill_mode='nearest'
)
```

मराठी अर्थ:

- `rescale=1./255` - पिक्सल व्हॅल्यू 0-255 वरून 0-1 मध्ये बदलते (नॉर्मलायझेशन)
- `validation_split=0.2` - 20% डेटा व्हॅलिडेशनसाठी, 80% ट्रेनिंगसाठी
- `rotation_range=20` - इमेजेस 20 अंशापर्यंत फिरवतो (ऑगमेंटेशन)
- `horizontal_flip=True` - इमेजेस आडव्या दिशेने उलटतो (मिरर इमेज)
- `zoom_range=0.2` - इमेजेस 20% पर्यंत झूम करतो
- `fill_mode='nearest'` - रोटेशन/झूम नंतर रिकाम्या जागा nearest पिक्सेलने भरतो

Training Generator:

```
train_generator = train_datagen.flow_from_directory(
    DATASET_PATH,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='sparse',
    subset='training',
    classes=['Pepper_bell_Bacterial_spot', 'Pepper_bell_healthy'],
    shuffle=True
)
```

मराठी अर्थ:

- `flow_from_directory` - फोल्डरमधून थेट इमेजेस लोड करते
- `target_size=(128,128)` - सर्व इमेजेस 128x128 मध्ये resize
- `class_mode='sparse'` - लेबल्स 0,1,2... अशा integer स्वरूपात
- `subset='training'` - हा ट्रेनिंग डेटा आहे (80% डेटा)
- `classes=[...]` - फक्त हे दोन फोल्डर वापरा
- `shuffle=True` - प्रत्येक epoch नंतर इमेजेस रँडमली मिसळ

Output दिसेल:

```
Found 1981 images belonging to 2 classes.  
Found 494 images belonging to 2 classes.  
Classes: ['Pepper_bell_Bacterial_spot', 'Pepper_bell_healthy']
```

याचा अर्थ:

- **1981** इमेजेस ट्रेनिंगसाठी
- **494** इमेजेस व्हॅलिडेशनसाठी
- **0** = Bacterial spot (रोगट)
- **1** = Healthy (निरोगी)

चौथा भाग: Class Weights (क्लास वेट्स)

```
all_labels = []  
for i in range(len(train_generator)):  
    _, labels = train_generator[i]  
    labels = labels.astype(int)  
    all_labels.extend(labels)
```

मराठी अर्थ:

- सर्व ट्रेनिंग इमेजेसची लेबल्स गोळा करतो
- `labels.astype(int)` - लेबल्स integer मध्ये बदलतो (0,1)
- हे लेबल्स एका लिस्टमध्ये साठवतो

```
unique_classes = np.unique(all_labels)  
print(f"Unique classes found: {unique_classes}")  
print(f"Class distribution: {np.bincount(all_labels)}")
```

Output:

```
Unique classes found: [0 1]  
Class distribution: [798 1183]
```

- 798 इमेजेस - Bacterial spot (रोगट)
- 1183 इमेजेस - Healthy (निरोगी)
- **डेटा असंतुलित आहे!** (Healthy जास्त आहेत)

```
class_weights = compute_class_weight('balanced',  
classes=unique_classes, y=all_labels)  
class_weight_dict = {i: class_weights[i] for i in  
range(len(class_weights))}
```

मराठी अर्थ:

- `compute_class_weight('balanced')` - असंतुलित डेटा साठी weights कॅल्क्युलेट करते
- फॉर्म्युला: $\text{weight} = \text{total_samples} / (\text{n_classes} \times \text{class_count})$
- Bacterial spot (class 0) ला जास्त weight मिळेल (कारण कमी इमेजेस)
- Healthy (class 1) ला कमी weight मिळेल (कारण जास्त इमेजेस)

Output:

Class weights: {0: 1.241, 1: 0.837}

- Class 0 (रोगट) 24% अधिक महत्वाचा
- Class 1 (निरोगी) 16% कमी महत्वाचा

पाचवा भाग: CNN Model Architecture (सीएनएन मॉडेल आर्किटेक्चर)

Block 1 (पहिला ब्लॉक):

```
layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2,2)),  
layers.Dropout(0.25),
```

मराठी अर्थ:

- `Conv2D(32, (3,3))` - 32 फिल्टर्स, प्रत्येक 3×3 साइजचा. **हे काय करते?** - इमेजमधून edges, corners, textures शोधते
- `activation='relu'` - नकारात्मक संख्या 0 करते. **का?** - non-linearity आणते (नेटवर्कला complex patterns शिकायला मदत)
- `BatchNormalization()` - प्रत्येक बॅच नंतर आउटपुट normalize करते. **फायदा:** त्वरीत training, स्थिरता
- `MaxPooling2D((2,2))` - 2×2 च्या विंडोमधून सर्वात मोठी व्हॅल्यू निवडते. **काय होते?** - इमेजचा साइज अर्धा होतो (128→64)
- `Dropout(0.25)` - 25% न्यूरॉन्स रँडमली बंद करते. **उद्देश:** ओव्हरफिटिंग टाळणे

Block 2 (दुसरा ब्लॉक):

```
layers.Conv2D(64, (3,3), activation='relu'),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2,2)),  
layers.Dropout(0.25),
```

- आता 64 फिल्टर्स (मागे 32 होते)
- इमेजचा साइज: 64×64 वरून 32×32

Block 3 (तिसरा ब्लॉक):

```
layers.Conv2D(128, (3,3), activation='relu'),  
layers.BatchNormalization(),  
layers.MaxPooling2D((2,2)),  
layers.Dropout(0.25),
```

- 128 फिल्टर्स (जास्त फीचर्स शिकते)
- इमेजचा साइज: 32×32 वरून 16×16

Global Pooling:

```
layers.GlobalAveragePooling2D(),
```

- प्रत्येक फीचर मॅपची सरासरी काढते
- **फायदा:** ओव्हरफिटिंग कमी करते, पॅरामीटर्स कमी करते

Dense Layers (फुली कनेक्टेड लेयर्स):

```
layers.Dense(128, activation='relu'),
layers.Dropout(0.5),
layers.Dense(64, activation='relu'),
layers.Dropout(0.3),
layers.Dense(2, activation='softmax')
```

- `Dense(128)` - 128 न्यूरॉन्स, सर्व फीचर्स एकत्र करतात
- `Dropout(0.5)` - 50% न्यूरॉन्स बंद (मजबूत regularization)
- `Dense(64)` - 64 न्यूरॉन्स
- `Dense(2, activation='softmax')` - आउटपुट लेयर, 2 क्लासेस. `softmax` probability वितरीत करते (एकूण = 1)

Model Parameters:

Total params: 119,042 (465 KB)
Trainable params: 118,594 (463 KB)

- सुमारे 1.19 लाख पॅरामीटर्स
- हे मध्यम आकाराचे model आहे (खूप मोठे नाही, खूप लहान नाही)

सहावा भाग: Compile Model (मॉडेल कंपाइल करणे)

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

मराठी अर्थ:

- `optimizer='adam'` - weights update करण्याची अल्गोरिदम.
 - **Adam म्हणजे काय?** - Adaptive Moment Estimation. ही सर्वात लोकप्रिय optimizer आहे.
 - **कसे काम करते?** - प्रत्येक पॅरामीटरसाठी स्वतंत्र learning rate ठेवते
- `loss='sparse_categorical_crossentropy'` - एरर कॅल्क्युलेट करण्याची पद्धत.
 - **स्पार्स म्हणजे काय?** - लेबल्स integer स्वरूपात आहेत (0,1) one-hot नाहीत
 - **क्रॉसएन्ट्रोपी म्हणजे काय?** - predicted probability आणि actual label मधील फरक मोजते
 - उदा: actual=1, predicted=0.9 → loss कमी; actual=1, predicted=0.1 → loss जास्त
- `metrics=['accuracy']` - प्रशिक्षण दरम्यान अचूकता मोजत राहते

सातवा भाग: Callbacks (कॉलबॅक्स)

```
callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5,
    restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss',
    factor=0.5, patience=3),
    tf.keras.callbacks.ModelCheckpoint('best_plant_disease_model.h5',
    monitor='val_accuracy', save_best_only=True)
]
```

मराठी अर्थ:

1. EarlyStopping (लवकर थांबवणे):

- `monitor='val_loss'` - व्हॅलिडेशन लॉस बघत राहा
- `patience=5` - जर 5 epochs मध्ये सुधारणा नाही झाली तर थांबा
- `restore_best_weights=True` - सर्वात चांगले weights परत आणा (जेव्हा loss सर्वात कमी होता)
- **उद्देश:** ओव्हरफिटिंग टाळणे आणि वेळ वाचवणे

2. ReduceLROnPlateau (लर्निंग रेट कमी करणे):

- `monitor='val_loss'` - व्हॅलिडेशन लॉस बघा
- `factor=0.5` - learning rate अर्धी करा (उदा. 0.001 → 0.0005)
- `patience=3` - जर 3 epochs मध्ये सुधारणा नाही झाली तर LR कमी करा
- **उद्देश:** minima मध्ये अडकून न राहण्यासाठी

3. ModelCheckpoint (मॉडेल सेव्ह करणे):

- `'best_plant_disease_model.h5'` - फाइल नाव
- `monitor='val_accuracy'` - व्हॅलिडेशन अक्युरसी बघा
- `save_best_only=True` - फक्त सर्वात चांगले model सेव्ह करा
- **उद्देश:** सर्वोत्तम model नंतर वापरण्यासाठी सेव्ह करणे



आठवा भाग: Model Training (मॉडेल प्रशिक्षण)

```
EPOCHS = 20
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=EPOCHS,
    callbacks=callbacks,
    class_weight=class_weight_dict,
    verbose=1
)
```

Training Process ची स्टेप बाय स्टेप माहिती:

Epoch 1 (पहिला फेरी):

Epoch 1/20
124/124 ————— - accuracy: 0.7667 - loss: 0.4695

val_accuracy: 0.5972 - val_loss: 0.6579

- **124/124** - 124 batches प्रोसेस झाले (1981 images / 16 batch size $\approx$ 124)
- **accuracy: 0.7667** - ट्रेनिंग डेटावर 76.67% अचूकता
- **loss: 0.4695** - ट्रेनिंग एरर (कमी चांगले)
- **val_accuracy: 0.5972** - व्हॅलिडेशन डेटावर 59.72% (नवीन इमेजेस)
- **val_loss: 0.6579** - व्हॅलिडेशन एरर

काय झाले? मॉडेलने पहिल्याच प्रयत्नात 76% ट्रेनिंग अचूकता मिळवली. पण व्हॅलिडेशन फक्त 60% आहे. हे सामान्य आहे कारण मॉडेलने अजून generalize शिकलेले नाही.

Epoch 5:

Epoch 5/20

accuracy: 0.9778 - loss: 0.0698

val_accuracy: 0.9453 - val_loss: 0.1287

- **97.78%** ट्रेनिंग अचूकता
- **94.53%** व्हॅलिडेशन अचूकता
- मॉडेल चांगले शिकत आहे! Train आणि Val मधील gap कमी आहे.

Epoch 11 (सर्वोत्तम):

Epoch 11/20

val_accuracy: 0.9939 - val_loss: 0.0452

- **99.39%** व्हॅलिडेशन अचूकता!
- हे खूप चांगले आहे

Epoch 15-20 (Overfitting सुरू):

Epoch 15: val_accuracy: 0.9939

Epoch 16: val_accuracy: 0.9939

Epoch 17: val_accuracy: 0.9919

Epoch 20: early stopping

- व्हॅलिडेशन अचूकता सुधारत नाही
- EarlyStopping ने 5 epochs नंतर थांबवले
- **Epoch 15 चे weights** परत आणले (सर्वोत्तम)

Training Output Summary:

Epoch	Train Acc	Val Acc	Learning Rate	Status
1	76.67%	59.72%	0.001	सुरुवात
5	97.78%	94.53%	0.0005	LR कमी झाला
11	99.34%	99.39%	0.0005	सर्वोत्तम
15	99.14%	99.39%	0.00025	Best saved

Epoch	Train Acc	Val Acc	Learning Rate	Status
20	99.14%	99.19%	0.000125	Early stopped

नववा भाग: Evaluation (मूल्यांकन)

```
val_loss, val_accuracy = model.evaluate(val_generator, verbose=0)
print(f"Validation Loss: {val_loss:.4f}")
print(f"Validation Accuracy: {val_accuracy:.4f}")
```

Output:

```
Validation Loss: 0.0137
Validation Accuracy: 0.9980
```

मराठी अर्थ:

- **Validation Loss: 0.0137** - खूप कमी एरर (शून्याच्या जवळ). परिपूर्ण!
- **Validation Accuracy: 99.80%** - 494 पैकी फक्त 1-2 इमेजेस चुकीचे वर्गीकरण

याचा अर्थ:

- तुमचे मॉडेल 99.8% अचूक आहे
- 100 पैकी 99.8 इमेजेस बरोबर ओळखते
- हे खूप उत्तम परिणाम आहेत!

दहावा भाग: Plots (आलेख)

Accuracy Plot:

```
ax1.plot(history.history['accuracy'], 'b-', label='Training')
ax1.plot(history.history['val_accuracy'], 'r-', label='Validation')
```

- **निळी रेषा (Training Accuracy)** - वर जाते (76% → 99%)
- **लाल रेषा (Validation Accuracy)** - वर जाते पण थोडी मागे
- Epoch 11 नंतर दोन्ही रेषा जवळ येतात

Loss Plot:

```
ax2.plot(history.history['loss'], 'b-', label='Training')
ax2.plot(history.history['val_loss'], 'r-', label='Validation')
```

- **निळी रेषा (Training Loss)** - कमी होते (0.47 → 0.01)
- **लाल रेषा (Validation Loss)** - कमी होते पण वर-खाली होते
- चांगले मॉडेल: दोन्ही रेषा खाली जातात

अकरावा भाग: Test Prediction (चाचणी भविष्यवाणी)

```
def test_prediction(generator, model, class_names, num_samples=3):
    images, labels = next(iter(generator))
    for i in range(min(num_samples, len(images))):
        prediction = model.predict(np.expand_dims(images[i], axis=0),
        verbose=0)
        predicted_class = class_names[np.argmax(prediction[0])]
        confidence = np.max(prediction[0])
        true_class = class_names[int(labels[i])]
        # Display image with prediction
```

मराठी अर्थ:

1. कॅलिडेशन जनरेटरमधून 3 इमेजेस घेते
2. प्रत्येक इमेजवर prediction करते
3. Predicted class vs True class तुलना करते
4. Confidence (आत्मविश्वास) percentage मध्ये दाखवते

Output:

```
True: Pepper_bell_Bacterial_spot
Pred: Pepper_bell_Bacterial_spot
Conf: 100.00%
```

याचा अर्थ:

- मॉडेल 100% आत्मविश्वासाने बरोबर उत्तर देत आहे
- सर्व 3 इमेजेस बरोबर वर्गीकृत झाल्या

बारावा भाग: Saved Files (सेव्ह केलेल्या फाइल्स)

```
model.save('plant_disease_model_final.h5')
model.save('plant_disease_model_final.keras')
```

तयार झालेल्या फाइल्स:

1. `best_plant_disease_model.h5` - सर्वोत्तम model (Epoch 15)
2. `plant_disease_model_final.h5` - अंतिम model (HDF5 फॉर्मॅट)
3. `plant_disease_model_final.keras` - अंतिम model (Keras फॉर्मॅट)

.h5 vs .keras:

- `.h5` - जुन्या पद्धतीचे (legacy) फॉर्मॅट
- `.keras` - नवीन recommended फॉर्मॅट

Final Results Summary (अंतिम निकाल)

```
Final Results:
Model Type: Custom CNN
Image Size: 128x128
Classes: Pepper_bell_Bacterial_spot, Pepper_bell_healthy
```

Validation Accuracy: 0.9980
Best Accuracy: 0.9939

मराठी सारांश:

- **मॉडेल प्रकार:** स्वतःचे बनवलेले CNN (Custom CNN)
- **इमेज साइज:** 128×128 पिक्सेल
- **एकूण इमेजेस:** 2,475 (1,981 train + 494 validation)
- **वर्ग:** रोगट मिरची आणि निरोगी मिरची
- **अंतिम अचूकता:** 99.80%! 🎉
- **सर्वोत्तम अचूकता:** 99.39%

Key Learnings (महत्त्वाचे मुद्दे)

1. Data Augmentation का महत्त्वाचे आहे?

- तुमच्याकडे फक्त 2,475 इमेजेस आहेत
- Augmentation ने 10x पटीने वैविध्य निर्माण केले
- त्यामुळे overfitting टळले

2. Class Weights का लागले?

- Healthy इमेजेस (1183) जास्त, Bacterial spot (798) कमी
- वजन (weights) दिल्याने मॉडेलने दोन्हीकडे समान लक्ष दिले

3. BatchNormalization चा उपयोग काय?

- Training ला स्थिरता आली
- Learning rate जास्त ठेवता आला
- Gradient vanishing problem सोडवली

4. EarlyStopping ने काय केले?

- Epoch 20 पूर्ण होण्याआधीच थांबवले
- Overfitting होण्यापासून वाचवले
- वेळ वाया घालवला नाही (20 ऐवजी 15 epochs)

5. Dropout चा उपयोग काय?

- Block 1-3 मध्ये 25% dropout
- Dense layers मध्ये 50% आणि 30% dropout
- यामुळे मॉडेल specific features वर जास्त अवलंबून राहिले नाही

Conclusion (निष्कर्ष)

तुमचे मॉडेल **99.8% अचूकता** सह कार्यरत आहे! हे परिणाम उद्योग स्तरावर वापरण्याजोगे आहेत.

प्राप्त केलेली कौशल्ये:

-  इमेज क्लासिफिकेशन CNN तयार करणे
-  डेटा ऑगमेंटेशन

- ☒ असंतुलित डेटा हाताळणे (class weights)
- ☒ कॉलबॅक्सचा वापर (EarlyStopping, ReduceLR, Checkpoint)
- ☒ मॉडेल सेव्ह करणे आणि लोड करणे
- ☒ भविष्यवाणी प्रणाली तयार करणे

तुम्ही आता हे मॉडेल Streamlit किंवा FastAPI वापरून deploy करू शकता! 🚀🌱

In []: